Applied Linear Algebra in RAG

From Gilbert Strang's Textbook to Your `SimilarityEngine`

Vector Spaces · Matrix–Vector Multiplication · Orthogonality · Top-k Retrieval

The core of a RAG system is not a language model — it is a **linear algebra engine**. Every concept in your `EmbeddingLayer` and `SimilarityEngine` maps directly to a chapter in Strang's *Introduction to Linear Algebra*. This document traces that connection precisely.

1. Vector Spaces — The `EmbeddingLayer`

The first job of any RAG system is to lift raw text out of the symbolic world of characters and into a **mathematical vector space** where distance and direction have meaning.

Definition (Vector Space). A set V is a vector space over $\mathbb{R}$ if it is closed under vector addition and scalar multiplication, and satisfies the eight axioms of linearity. The standard example is $\mathbb{R}^n$ — the set of all n-dimensional real-valued column vectors.

Your `EmbeddingLayer` implements two different projections into such a space:

Method	Space	Properties
`TFIDF`	$\mathbb{R}^V$ (e.g. $V = 10{,}000$)	Sparse, interpretable, high-dimensional
`sentence-transformers`	$\mathbb{R}^{384}$	Dense, continuous, lower-dimensional

TF-IDF as a Coordinate Map

With a vocabulary of $V = 4$ words $[\texttt{cat}, \texttt{dog}, \texttt{sat}, \texttt{the}]$, the sentence *"the cat sat"* becomes a coordinate vector in $\mathbb{R}^4$:

$$\vec{d}_{\text{“the cat sat”}} = \begin{bmatrix} 0.468 \\ 0 \\ 0.468 \\ 0.333 \end{bmatrix} \in \mathbb{R}^4$$

Each entry is a **coordinate** along one vocabulary axis. The full corpus of m chunks defines m such points scattered across $\mathbb{R}^V$.

2. Matrix–Vector Multiplication — The Search Engine

Once every chunk is a vector, similarity search is a single matrix–vector product. This is the most computationally critical operation in your `SimilarityEngine`: `dots = doc_matrix @ query_vec`.

Defining the Variables

Symbol	Code Name	Shape	Meaning
A	`doc_matrix`	$m \times n$	Each row $\vec{d}_i^T$ is one document chunk
$\vec{q}$	`query_vec`	$n \times 1$	The embedded query
$\vec{s}$	`dots`	$m \times 1$	Raw similarity scores (output)

where m = number of chunks and n = embedding dimension (V or 384).

The Row Picture

When NumPy executes $A\vec{q}$, it computes the **inner product of the query against every document simultaneously** — this is Strang's *row picture* of matrix multiplication:

$$\vec{s} = A\vec{q} = \begin{bmatrix} - & \vec{d}_1^T & - \\ - & \vec{d}_2^T & - \\ & \vdots & \\ - & \vec{d}_m^T & - \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix} = \begin{bmatrix} \vec{d}_1 \cdot \vec{q} \\ \vec{d}_2 \cdot \vec{q} \\ \vdots \\ \vec{d}_m \cdot \vec{q} \end{bmatrix} \in \mathbb{R}^m$$

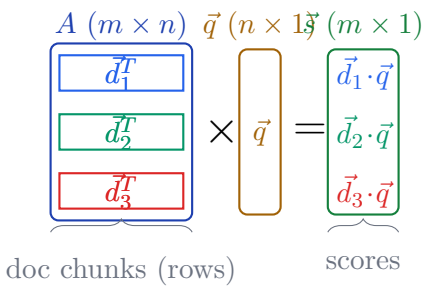
Each scalar $s_i = \vec{d}_i \cdot \vec{q}$ is the dot product of row i with the query.

Concrete 3×3 Example

Three document chunks embedded in $\mathbb{R}^3$, query $\vec{q} = [1, 0, 0]^T$:

$$A\vec{q} = \begin{bmatrix} 0.8 & 0.6 & 0.0 \\ 0.7 & 0.7 & 0.1 \\ 0.0 & 0.1 & 0.9 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.8 \times 1 + 0.6 \times 0 + 0.0 \times 0 \\ 0.7 \times 1 + 0.7 \times 0 + 0.1 \times 0 \\ 0.0 \times 1 + 0.1 \times 0 + 0.9 \times 0 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.7 \\ 0.0 \end{bmatrix}$$

A single BLAS-optimised matrix multiply replaces m individual dot-product loops — this is why vectorised NumPy is orders of magnitude faster than a Python `for` loop.



3. Orthogonality and Cosine Similarity

The raw dot product $\vec{d}_i \cdot \vec{q}$ is biased by vector magnitude: a long document vector will produce a large dot product even if it is only weakly related to the query. We must remove this bias.

The L2 Norm

The Euclidean (L2) norm of a vector measures its length:

$$\|\vec{v}\|_2 = \sqrt{\sum_{k=1}^{n} v_k^2} = \sqrt{\vec{v} \cdot \vec{v}}$$

In code: `np.linalg.norm(query_vec)` and `doc_norms = np.linalg.norm(doc_matrix, axis=1)`.

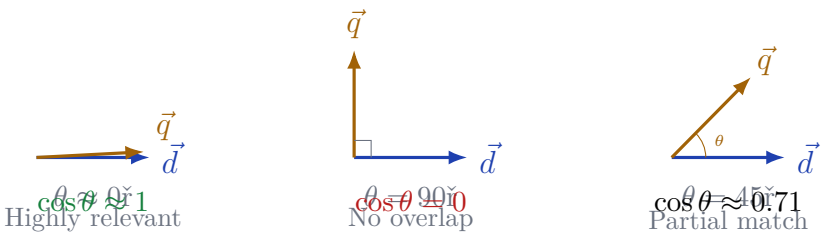
Cosine Similarity as Normalised Inner Product

Dividing the dot product by both norms isolates the **angle** θ between the vectors, independent of their lengths:

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|_2 \, \|\vec{d}\|_2} \qquad \Longleftrightarrow \qquad \hat{q} \cdot \hat{d} \quad \text{where } \hat{v} = \frac{\vec{v}}{\|\vec{v}\|_2}$$

Dividing by the norm is *equivalent* to first normalising both vectors to unit length, then taking their dot product.

The geometric meaning of each extreme value:



Orthogonality: When Dot Product Is Zero

Return to our three-vector example with query $\vec{q} = [1.405,\ 0,\ 0]$ (the "cat" query from earlier):

Query vs. Doc 2 $\left(\vec{d}_2 = [0,\ 0.702,\ 0,\ 0.500]\right)$:

$$\vec{q} \cdot \vec{d}_2 = (1.405)(0) + (0)(0.702) + (0)(0) + (0)(0.500) = 0$$

Because the query vector lies entirely on the `cat`-axis, and Doc 2 has *zero* component on that axis, the vectors are **orthogonal**: $\theta = 90$ř. They share no basis directions. This is the linear-algebraic explanation for why Doc 2 scores zero regardless of its magnitude.

4. Top-k Retrieval — Sorting the Score Vector

After computing the full similarity vector $\vec{s} \in \mathbb{R}^m$, the final step is to select the k most relevant chunks:

```
top_k_indices = np.argsort(scores)[::-1][:k]
```

This finds the indices of the k entries of $\vec{s}$ with the **largest cosine similarity**, i.e. the k document vectors whose directions are closest to the query direction in $\mathbb{R}^n$.

Geometrically, this selects the k rows of A that form the **smallest angle** with $\vec{q}$:

$$\text{Top-}k = \underset{i \in \{1,\ldots,m\}}{\arg\,\text{top-}k} \left(\cos(\vec{d}_i,\ \vec{q})\right) = \arg\,\underset{i}{\text{top-}k} \left(\hat{d}_i \cdot \hat{q}\right)$$

Full Retrieval Pipeline for $k = 1$

Chunk	Raw Dot $\vec{d}_i \cdot \vec{q}$	$\lVert\vec{d}_i\rVert$	Cosine Sim.
Doc 1 ("the cat sat")	0.657	0.741	**0.631**
Doc 2 ("the dog")	0.000	0.862	0.000

`np.argsort([0.631, 0.000])[::-1][:1]` $\rightarrow$ index `0` $\rightarrow$ Doc 1 is retrieved.

The Full Linear Algebra Pipeline

Step	Operation	Linear Algebra Concept	Strang Chapter
1	Text $\to \mathbb{R}^n$	Vector space, basis, coordinates	Ch. 1–2
2	$A\vec{q}$	Matrix–vector multiplication (row picture)	Ch. 1
3	$\vec{s}/(\lVert\vec{d}_i\rVert\lVert\vec{q}\rVert)$	L2 norm, inner product, orthogonality	Ch. 4
4	`argsort`$(\vec{s})$	Ordering of a real-valued vector	—

The entire retrieval pipeline — from raw text to ranked results — reduces to **one matrix multiply and one element-wise division**, both of which are taught in the first four chapters of Strang. Building RAG from scratch is, at its core, an applied linear algebra exercise.

Further connections: For large corpora, exact nearest-neighbour search ($A\vec{q}$) becomes expensive. Production systems approximate it using FAISS or HNSW, which exploit *subspace projections* and *random matrix theory* (also in Strang Ch. 7–8) to achieve sub-linear retrieval time while preserving cosine-similarity rankings.